

Агент в реальном проекте: изоляция, контекст, оркестрация

Конспект-презентация по тому, что было в эфире. Таймкоды — по записи.
Неточности, прозвучавшие вживую, поправлены и помечены.

Что было на самом деле

ЗАПИСЬ	БЛОК	МИН	ГЛАВНОЕ
до записи	Разбор домашних заданий	~12	Правила в работах написаны словами — смотрим, чего они стоят
00:00	Prompt injection	9	Живое демо: Naiku против скрытой инструкции, permissions, обход запрета
09:11	Безопасность и изоляция	4	Лестница песочниц: встроенная → ОС → VM / Docker
13:34	Контекст, оптимизаторы, память, индексы	17	rtk, context-mode, голос, память, GitNexus и Graphify на element-web
30:13	Оркестрация	19	Субагенты, замер «параллельно против последовательно», worktrees, workflows, ADB / WDA
50:38	Вопросы и ответы	16	Домашка, команды, Superpowers, где следить за новинками
66:40	Бонус для оставшихся	24	Pencil, кросс-проверка моделями, секреты, локальные модели, 3D-аватар

Почему словесный запрет — не механизм

На первом занятии агент удалил папку Temp, хотя в CLAUDE.md было написано не удалять. Разобрали как «формулировка была мягкой».

Это половина причины. Вторая половина: **словесный запрет работает ровно до первого текста, который окажется убедительнее вашего**. А такой текст может прийти откуда угодно — со страницы, из файла, из письма клиента.

Отсюда план занятия: инъекция объясняет *почему*, дальше три ответа — *чем заменить*.

Четыре опоры доверия — второй виток

ОПОРА	ЗАНЯТИЕ 1	ЗАНЯТИЕ 2
Ограниченность	фраза в CLAUDE.md	песочница, права ОС, хук
Предсказуемость	файл правил	что агент помнит и не помнит
Компетентность	скилл под стек	индекс чужого кода, агенты-специалисты
Проверяемость	один аудитор	несколько независимых моделей

БЛОК 1 · 00:00 — 09:10

Prompt injection

В контекст модели попадает практически всё: инструменты, прочитанные файлы, ваши промпты, её ответы. Если в прочитанном была спрятана инструкция — она тоже там.

Слабая модель против спрятанной инструкции

Полигон. Файл-каталог товаров. Внутри — «системная» инструкция: открыть Google, найти курс доллара, сообщить пользователю.

Запрос. Обычный, пользовательский: «Соберите корзину на 5000 рублей».

Модель. Haiku, минимальный effort — чтобы у атаки был шанс. Современные сильные модели такое отбивают сразу.

Позже тот же файл отдан сильной модели: она нашла **четыре** инъекции, глазом на экране видно три.

Результат: инъекция не сработала

Haiku **молча** не выполнила инструкцию — остановили guardrails самой модели. Сильная модель — **назвала** проблему вслух.

Почему это важнее успешного взлома

«Молча не сделала» и «сказала вслух» — **разные уровни защиты**. Полагаться можно только на второй: первый ничего не сообщает, и вы не узнаете, что атака вообще была.

Где прячут инструкции

- белым шрифтом по белому фону на странице;
- в HTML-тегах, комментариях, метаданных файла;
- под видом системных сообщений и служебных тулзов;
- в поле, куда пишет посторонний человек: назначение платежа, комментарий к переводу, обращение в поддержку;
- **в описании инструмента** — скилла или MCP-сервера, который вы скачали. Описание попадает в контекст как доверенный текст.

Последнее — отдельный класс атак, *tool poisoning*: инструмент вёл себя честно, обновился и сменил поведение.

«Штука опасная, потому что может вызвать вообще любой запрос, который мошенник придумает. Это просто обычный взлом.»

Из эфира, 02:00

Для финтеха

Поле «назначение платежа», текст обращения клиента, скачанный тариф конкурента — это **недоверенный текст, который попадает в промпт каждый день**. Инъекция здесь не учебная выдумка.

Permissions: `allow`, `ask`, `deny`

Файл `~/.claude/settings.json`, секция `permissions`.
Базовые правила того, что агенту можно и что нельзя.
Настраивается без знания программирования — проще всего попросить самого агента.

Порядок проверки всегда один

`deny` → `ask` → `allow`. Первое совпадение решает.
Широкий `deny` перекрывает узкий `allow` —
исключений из запрета не бывает.

Режимы: `default` (manual) → `acceptEdits` → `auto` → `dontAsk` →
`bypassPermissions`. Документация: code.claude.com/docs/en/permissions

УТОЧНЕНИЕ К ЭФИРУ

В эфире: «в `bypass permissions` не работают»

Точнее так: `bypassPermissions` отключает
подтверждения — правила `allow` и `ask` теряют
смысл. Но **`deny` -правила действуют во всех
режимах**, включая `bypass`. Документация: «explicit
deny rules still apply».

Вместе с ними в `bypass` остаются: блокирующие
`PreToolUse` -хуки и защита критических путей от `rm`.

Где `deny` протекает

`deny` — это **шаблон на строку команды**. Он не понимает, что делает команда, он сравнивает текст.

ВЫЗОВ	ЧТО ПРОИЗОШЛО
<code>cat secrets.txt</code>	запрещён — <code>deny</code> сработал
тот же файл другой командой	прочитан — шаблон не совпал
<code>python -c "open('secrets.txt')..."</code>	прочитан — на это запрета нет

Перечислять все способы прочитать файл — бесконечное множество. Разрешать нужно конечное.

Что не обойти сменой команды

- **Песочница** — ограничение на уровне ядра ОС, не читает команду, а режет доступ к файлам и сети;
- **PreToolUse -хук** — код, который выполняется до инструмента и может его запретить. Ничего не читает, ни в чём не убеждается;
- **Права ОС** на файл с секретами — агент не получит к нему доступа, что бы ни случилось.

Хук был обещан «ближе к концу занятия» и до него не дошли — он будет на занятии 3.

БЛОК 2 · 09:11 – 13:30

Изоляция

Если списка правил мало — нужен механизм, который не читает и не убеждается. Три ступени, от встроенной песочницы до отдельной машины.

Три уровня изоляции

1

Встроенная песочница Claude Code — команда `/sandbox`

Накрывает **только Bash и его дочерние процессы**. macOS — системный Seatbelt, Linux — bubblewrap. Два режима: *auto-allow* (команды внутри без вопросов) и *regular permissions*. По умолчанию запись — только в рабочую папку, каждый новый сетевой домен — с подтверждением.

2

Контейнер или виртуальная машина

Docker-образ с агентом внутри, либо чистая VM. Ограничение на уровне системы: всё, что агент сделает внутри, внутри и останется. Для сомнительного кода, чужих репозиториев, работы без присмотра.

3

Отдельная чистая машина

«Самый надёжный вариант — запускать сомнительный проект, сомнительное задание или research на отдельной чистой машине.»

Документация: code.claude.com/docs/en/sandboxing · .../sandbox-environments

Что песочница *не* покрывает

MCP и обвязка остаются снаружи

Прозвучало в эфире и подтверждается документацией: песочница «применяется только к Bash-командам и их дочерним процессам». MCP-серверы, AppleScript, инструменты через UI — всё это продолжает дотягиваться туда, куда терминалу уже нельзя.

Отдельный уровень защиты для них нужно ставить дополнительно. Сам Claude Code не даёт sandboxed-команде править `.mcp.json`, хуки и настройки — иначе команда могла бы выдать себе права.

УТОЧНЕНИЕ К ЭФИРУ

«Apple считает свой sandbox deprecated»

Deprecated помечена утилита `sandbox-exec`. Сам механизм Seatbelt жив, на нём держится App Sandbox, и именно его использует встроенная песочница Claude Code на macOS. Пользоваться можно.

Минимум на понедельник

- не `bypassPermissions`, а хотя бы `auto` или `manual`;
- периодически перечитывать свой `settings.json`;
- песочница там, где сомневаешься в источнике.

Контекст, оптимизаторы, память, индексы

Механизм ставит границу. Внутри границы агент должен ещё и не задыхаться.

Почему он кончается и что с этим делать

Контекст — это всё сразу: системный промпт, описания инструментов, прочитанные файлы, ваши сообщения, ответы агента, выводы команд. Он заполняется, и дальше два пути: новая сессия или компакт.

«Не обязательно, что результат будет плохим. Просто если что-то пойдёт не так, починить это сложнее, чем начать новую сессию.»

14:00

Чистка контекста безопасна

Файлы в папке не трогаются — это всё внутри чата. При хорошо настроенном проекте потеря контекста не страшна: новый чат, «подтяни всё, что знаешь о проекте» — и дальше.

Оптимизаторы, показанные в эфире

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	ЗВЁЗД
rtk	CLI-прокси: перехватывает команды и отдаёт агенту сжатый вывод	77 тыс.
context-mode	выводы инструментов не идут в контекст — остаётся результат	20 тыс.

Честная цифра

«Заявляют 99 %. Реально освобождается процентов 20, может быть 30.» Хорошая модель сама закрывает базовый context management — оптимизатор лишь немного экономит токены.

github.com/rtk-ai/rtk · github.com/mksglu/context-mode · звёзды на 22.08.2026

Как искать и проверять инструменты

1

Сформулировать

Что нужно или что хочется сделать — и добавить к запросу слово GitHub. Почти всё живёт там.

2

Спросить агента

Скинуть ссылку: «расскажи про репозиторий». Понять, нужно ли оно вообще.

3

Проверить отдельно

«Проверь репозиторий: что ставит, какие хуки, куда ходит». Отдельным запросом, до установки.

4

Ставить

«Если безопасен — установи глобально». Агент либо поставит, либо спросит, если есть сомнения.

Ориентир по доверию

«Репозиторий на 20 тысяч звёзд и 2000 коммитов — вероятность, что туда попали инъекции, сильно ниже, чем у репозитория в 100 звёзд.» Звёзды — не гарантия, но много глаз.

Где искать

GitHub → Explore → Trending. Профильные телеграм-каналы и рилсы — GitNexus и Graphify были найдены именно так. Живой чат по агентам, платформам и инструментам — t.me/vibecoderchat. Проверять всё найденное обязательно.

Голос вместо клавиатуры

«Большие промpts руками писать вообще неприятно. Качество сильно выше, но очень долго.»

20:19

Показан **Typeless**: одна кнопка, говоришь — текст появляется у курсора. Что делает сверх распознавания:

- убирает слова-паразиты и задумчивость;
- повторённую мысль записывает один раз;
- Docker, Nginx, Swift пишет правильно внутри русского текста;
- умеет списки и перевод.

Что даёт на практике

Промpts становятся длиннее и содержательнее: вместо односложных «продолжай» — целый флоу задач за один раз. Время на промптинг падает в разы.

ВАРИАНТ	ОСОБЕННОСТЬ
Typeless	показан в эфире; macOS, Windows, iOS, Android
Wispr Flow	тот же класс, облачная обработка
Локальные на Whisper	бесплатно, ничего не уходит в облако, пара ГБ памяти; например OpenWhisper
Ollama	если собирать локальную модель самому

Что агент помнит между сессиями

В эфире агенту задан прямой вопрос: «Расскажи, как устроена твоя память». Ответ — это `.md`-файлы.

слой	что это
CLAUDE.md	ваши инструкции; читаются при старте, иерархия от домашней папки до подпапок
Auto memory	заметки, которые агент пишет сам по вашим поправкам: <code>~/.claude/projects/<проект>/memory/</code> , индекс <code>MEMORY.md</code>

Туда влезает немного — агент сам держит память чистой, кладёт выжимку.
Документация: code.claude.com/docs/en/memory

Если базовой памяти мало

claude-mem и подобные — хуки, которые сохраняют всё в персистентную память между сессиями. Крупных штук пять, выбирать по вкусу. github.com/thedotmack/claude-mem, 92 тыс. звёзд.

Вариант ведущего — Obsidian

Обычные заметки, куда агент пишет через хуки: daily-записи, состояние проектов. Причина — **не только Claude**: «чтобы другие модели сразу знали что-то о проекте». Потом это читает другой агент с тем же Obsidian.

Индекс кодовой базы

Полигон — **element-web**, open-source мессенджер на Matrix: 5 177 файлов, 452 тыс. строк TypeScript, 70 тыс. коммитов. Склонирован и запущен локально «в три промпта».

ИНСТРУМЕНТ ЧТО ДЕЛАЕТ

GitNexus	граф вызовов, blast radius изменений; вместо grep и read агент ходит по индексу. github, 46 тыс. звёзд
-----------------	--

Graphify	граф связей всего проекта, включая доки и схемы; индексируется дольше, граф больше. github
-----------------	--

Друг другу не мешают, работают вместе. Брать можно любой аналог.

Зачем

Чтобы модель понимала проект без лишних проходов по файлам. Один и тот же вопрос к коду («где считаются комиссии и кто это вызывает») до индекса и после отличается и по токенам, и по качеству.

«Если проект маленький — это вообще не нужно, абсолютно.»

30:00

«Придя в большой проект, не нужно сразу запускать туда ИИ. Сначала GitNexus или Graphify — чтобы он каждый раз кучу токенов не тратил.»

85:20

Оркестрация агентов

Один агент с идеальным контекстом всё равно один. Что такое субагент, когда параллельность выигрывает, а когда нет, и как оставить агента работать на ночь.

Что такое агент и что такое оркестрация

Агент

Та же модель, что в чате. Спавнится отдельно, работает в **своём контекстном окне** со своим системным промптом и возвращает в главную сессию только результат. Те же мозги, другие промпты. Вызвать может и человек, и другой агент.

Базовая оркестрация

Несколько субагентов под одной задачей: проверить репозиторий, проиндексировать проект, изучить три подсистемы. «Самый простой, удобный и начальный путь.» В Claude Code и Codex есть из коробки — просить словами.

«Это сейчас очень модное слово. „Я занимаюсь оркестрацией, у меня команда из 10 агентов, они делают всю работу за меня“. Это не всегда правда. Проверить работу 10 агентов — не то же самое, что проверить работу в одном чате.»

30:40

Документация: code.claude.com/docs/en/sub-agents

Параллельно против последовательно

Задача. «Изучи три подсистемы element-web» — один раз тремя субагентами параллельно, второй раз последовательно в одной сессии.

По времени: последовательно быстрее

Каждый субагент тянет свой системный промпт, загрузку инструментов и стартовые проходы, и не знает, что нашли другие. На небольшой задаче это дороже, чем сделать всё подряд.

Выигрыш по времени появляется, когда частей много и они по-настоящему независимы: «10 агентов — 10 research'ей за одну единицу времени».

КОНТЕКСТ ГЛАВНОЙ СЕССИИ ПОСЛЕ ПРОГОНА

128 тыс.

ТРИ СУБАГЕНТА · $\approx 15\%$ ОТ 1 МЛН

≈ 1 МЛН

ПОСЛЕДОВАТЕЛЬНО · СЕССИЯ ЗАБИТА

Главный эффект — не скорость, а чистый контекст

В главную сессию возвращаются только результаты — без «мусорных» чтений файлов и вызовов инструментов. Можно продолжать работу в той же сессии, где уже накоплен контекст задачи.

Worktrees

«Три субагента в одной сессии — это один человек, разделённый на три. Три агента в трёх worktrees — три независимых разработчика: своя git-история, свои коммиты, свои файлы, и потом отдельные pull-реквесты.»

40:00

В одной папке агенты спотыкаются друг о друга: файлы общие. В worktree у каждого своя ветка и своё окружение, merge — руками.

Запускается промптом: «работаем в worktrees, сделай три агента, каждый в своём». В Claude Code субагенту можно выдать worktree одной строкой — `isolation: worktree`.

Цена

Worktree разворачивает полную рабочую копию файлов (.git общий). На большом проекте с зависимостями три ветки — три node_modules.

«У меня один раз проект — 250 гигабайт чисто на worktrees. Диск заполнится, работа встанет. Если вас нет за компьютером — будет обидно.»

42:21

git-scm.com/docs/git-worktree

Workflow и ночной прогон

Workflow — большой промпт с шагами, сохранённый как повторяемый паттерн: скилл, хук, файл процедуры. «Инструкция для агента, которая выполняет по определённым шагам определённые действия.»

Схема, по которой работает ведущий

спецификация → 2 независимых ревьюера × 2 раунда
→ план → проверка плана × 2
→ реализация → тесты → аудит реализации агентами
→ финальная проверка независимым агентом → отчёт файлом

Вечером 15–20 минут на промпт по материалам дня — утром готовая, проверенная фича. Если установлен Superpowers — он сам предложит сохранить флоу как workflow.

13 ч

САМЫЙ ДОЛГИЙ АВТОНОМНЫЙ ПРОГОН

«Он сделал реально крутую фичу, очень большую, хорошо её проверил сам и всё мне выкатил.»

Условие

Потенциально опасная работа без присмотра — только в песочнице, с запретами. «Главное, чтобы у вас всё было настроено.»

ADB, WDA, Playwright

ПЛАТФОРМА	ИНСТРУМЕНТ	ССЫЛКА
Android	ADB — Android Debug Bridge	developer.android.com/tools/adb
iOS	WDA — WebDriverAgent	github.com/appium/WebDriverAgent
Веб	Playwright	playwright.dev
Бэкенд	проверяет сам — нужен только стенд	—

УТОЧНЕНИЕ К ЭФИРУ

WDA в эфире расшифрован как «Work Development Access» — правильно **WebDriverAgent**, WebDriver-сервер для iOS от проекта Appium.

«Можно подключить Android по ADB, и агент спокойно выполняет 80 % ручного тестирования. То же самое через iPhone — WDA. Даже удобнее симулятора: звонки, пуши — на симуляторе это просто не проверишь.»

47:30

Картина целиком

Ночью — код по workflow, затем агент прогоняет его на живом телефоне. Утром — рабочая задача, которую нужно провалидировать и прочитать код.

После основной части

Superpowers, кросс-проверка моделями, секреты, данные и обучение, локальные модели, Pencil. Сорок минут, из которых двадцать четыре — уже после «занятие закончилось».

Superpowers

276 тыс. ОКТ. 2025

ЗВЁЗД НА 22.08.2026

ПЕРВЫЙ КОММИТ

Обкатан людьми: много пользователей, десятки контрибьюторов — «аккумулированные данные многих людей и многих проблем, которые он решает».

github.com/obra/superpowers

Что делает

«Ставит вам хуки, через которые не пройти просто так. Забыли сделать спецификацию или продумать задачу глубже — он отловит и задаст вопросы. На этапе проектирования, будь то код или текстовое описание сайта.»

Почему это важно

«Чем раньше поправить мысли модели, тем лучше результат. Исправлять всегда сложнее, чем сразу сделать хорошо.» Это небольшая, но готовая агентная обвязка из коробки.

Спес-фреймворки, о которых спросили: SpecKit, OpenSpec. Ведущий ими не пользуется — ручной флоу на Superpowers.

Кросс-проверка несколькими моделями

«Оценку через ИИ я бы рекомендовал проводить минимум через трёх независимых агентов: Orus, Codex и третью модель. Дорого по токенам и по времени, но результат классный.»

71:50

Пример. Презентация для защиты MBA, сделанная через Codex. Прогнана через Orus и кросс через две другие модели, собрана воедино — зашла с первого раза.

«Доверять отдельной модели я бы не стал. Самый вопрос — корнер-кейсы, когда всё идёт чуть-чуть не так. Чтобы не ломалось — кросс-проверки и спецификации с планами.»

Наблюдение, которое ломает интуицию

Один сетап, один проект, одна линейка моделей — а поведение разное. На Orus 4.8 — 13 часов автономной работы; на Orus 5 рекорд — около четырёх: «у него как будто более серьёзные защитные механизмы, он останавливается спросить подтверждение».

Вывод: свой флоу нужно **тестировать на тестовом проекте** при каждой смене модели, а не переносить вслепую.

Это один подход. TDD и остальные паттерны никуда не ушли.

Как не слить ключи

«Агент в конечном итоге всегда может получить данные и прочитать их. В какой-то момент расслабитесь, он прочитает, ключ утечёт в чат — и он потом ещё будет рекомендовать его ротировать, а там бюрократия.»

83:30

Полгода назад массово шли инъекции на поиск сид-фраз блокчейн-кошельков — ведущий после этого «почистил весь компьютер от sensitive-данных». Широкие разрешения в его сетапе держатся на этом, и повторять это он не рекомендует.

1. Права на уровне ОС

Запретить чтение файла с ключами всем, кроме sudo. Агент не получит доступ, что бы ни случилось.

2. Плейсхолдеры

В рабочем файле — заглушка, реальное значение подставляется отдельно и живёт там, куда агент не ходит.

Перед сдачей любой работы

`.env` , `settings.json` , история команд, скриншоты терминала. Ключ утекает не через взлом, а через файл, который вы сами показали.

Учатся ли на ваших данных

УТОЧНЕНИЕ К ЭФИРУ

В эфире: «хорошая подписка говорит, что не учимся на ваших данных, тем более enterprise или API». На деле условия разные:

КАК ПОЛЬЗУЕТЕСЬ	ИСПОЛЬЗУЮТСЯ ЛИ ДЛЯ ОБУЧЕНИЯ
Free, Pro, Max — включая Claude Code под этими аккаунтами	Да, по умолчанию с сентября 2025 — пока не выключить «Help improve Claude» в настройках приватности
API, Claude for Work, Team, Enterprise	Нет — коммерческие условия

Источник: anthropic.com/news/updates-to-our-consumer-terms, privacy.claude.com

Что сделать сегодня

Если работаете по личной подписке Pro или Max — зайти в Privacy Settings и выключить «Help improve Claude». Это один переключатель.

«Они же могут и врать. Один-два иска на копейки, когда они обучат модель и будут отбивать это ещё 30 лет.»

86:00

Скепсис уместен. Но переключатель выключить стоит в любом случае — это бесплатно.

Китайские open-source-модели

Прямой ответ на вопрос «как не слить большой прод-проект»: своё железо, VPN, локальная модель. Claude Code и другие CLI умеют работать с локальной моделью вместо подписки — те же инструменты, безлимитные токены.

УТОЧНЕНИЕ К ЭФИРУ

Kimi K3 — цифры

В эфире: «27 миллиардов, триллион параметров». По релизу Moonshot (июль 2026): **2,8 трлн параметров МоЕ**, из них активны ≈104 млрд на токен, контекст 1 млн. Открытые веса. На консьюмерском железе не пойдёт — нужен сервер.

«Они все будут хуже Claude или ChatGPT — на 2, 3, 5, 10, максимум 15 %. Но 15 % хуже последнего Opus 5 — это Opus 4.5–4.7, который был 2–3 месяца назад, а это реально крутой уровень. Я на нём писал, и было классно.»

87:00

МОДЕЛЬ	КОММЕНТАРИЙ ИЗ ЭФИРА
Kimi K3 / K2	«код пишет отлично, агентов спавнит, всё делает»
GLM 5.2 / 5.3	«вышел недавно»
MiniMax	«тоже отличная»

Возможно, не такой умный — «сделайте чуть больше обвязки». Ссылка: platform.kimi.ai

Pencil и «ограничений практически нет»

Pencil — дизайн структурой, а не картинкой

Инструмент класса Figma с MCP-сервером: агент рисует экран заказа кофе вживую, результат лежит в .rep -файле и правится руками. «Супер для дизайнеров, соло-разработчиков, презентаций.»

pencil.dev

История про автора и Figma в эфире — по пересказу ведущего; проверить не удалось, приводим как анекдот.

Та же мысль шире: генерацию видео, картинок, озвучку — всё, что делали руками через ИИ, — удобнее делать через кодинговый агент: найти MCP под сервис, выдать доступ по API, он сам проверит результат.

«Переозвучил своё видео на английский своим голосом через Claude: он скачал voice-модель, клонировал голос, учёл тайминги и разницу длины речи. В итоге взял диктора — акцент прослеживался. Но опыт прикольный.»

61:20

«3D-аватар в мобильном приложении за два дня на React Native: взял модель из интернета, органы, шейдеры, анимации, прозрачности. Я в 3D никто, шейдеры писать не умею. Оказалось, можно практически всё.»

75:20

Что унести с занятия

1

Словесный запрет — не механизм

Он держится до первого убедительного текста.
deny — шаблон на строку.
Механизм — песочница, хук, права ОС.

2

Песочница накрывает Bash

Не MCP, не обвязку. Для сомнительного — контейнер или VM. Для работы без присмотра — обязательно.

3

Параллельно ≠ быстрее

Субагенты экономят контекст главной сессии, а не время. Время выигрывают только независимые большие задачи.

4

Процедура важнее промпта

Workflow, worktrees, ночной прогон, проверка на устройстве. И независимый аудит — другой моделью, в другой сессии.

Главный приём, который работает для всего остального: всё, чего не знаете, и всё, в чём сомневаетесь, — прогоняйте через самого агента. Настройки, безопасность, чужой репозиторий, свой флоу.

Три обязательных пункта и плюсы

Обязательно

- **Работающий результат** — продукт, функционал, исследование; небольшой, но живой. Текст — в PDF.
- **Агенты** — работа делается субагентами, это видно.
- **Workflow** — хотя бы один, под реальную задачу. Скилл, хук, файл, промпт — формат любой.

В плюс

- три worktree и merge;
- изоляция с доказательством отказа;
- инъекция на двух моделях;
- оптимизатор и индекс — с честным отчётом;
- замеры, журнал сессий;
- переход с архива на Git.

Соло или команда до 3–4 человек. Командам — плюс, межролевым — особенно: бэк + фронт + мобилка = целый сервис.

29.08

СУББОТА, 18:00

Ссылка на репозиторий — в телеграм [@oleg56x](#).
Приватный — доступ [@gfarhad93](#). Варианты и промпты — в [hw-02.pdf](#) и [hw-02-materials.pdf](#).

ССЫЛКИ

Всё, что упоминалось

ДОКУМЕНТАЦИЯ CLAUDE CODE

Permissions	code.claude.com/docs/en/permissions
Режимы разрешений	.../permission-modes
Песочница	.../sandboxing
Контейнеры и VM	.../sandbox-environments
Субагенты	.../sub-agents
Память	.../memory

ИНСТРУМЕНТЫ

Superpowers	github.com/obra/superpowers
rtk	github.com/rtk-ai/rtk
context-mode	github.com/mksglu/context-mode
GitNexus	github.com/abhigyanpatwari/GitNexus
Graphify	github.com/safishamsi/graphify

ЛИЧНЫЕ РЕКОМЕНДАЦИИ ВЕДУЩЕГО

YouTube · Telegram	@ProdAdvice · t.me/GitHubRadar
Чат вайбкодеров	t.me/vibecoderchat

ИНСТРУМЕНТЫ

claude-mem	github.com/thedotmack/claude-mem
Obsidian	obsidian.md
Context7	github.com/upstash/context7
Typeless · Wispr Flow	typeless.com · wisprflow.ai
Ollama	ollama.com
Pencil	pencil.dev
SpecKit · OpenSpec	github.com/spec-kit · Fission-AI/OpenSpec
element-web	github.com/element-hq/element-web

ТЕСТИРОВАНИЕ

ADB · WDA · Playwright	adb · WebDriverAgent · playwright.dev
------------------------	--

ПОЛИТИКИ И МОДЕЛИ

Данные для обучения	anthropic.com/news/updates-to-our-consumer-terms
Kimi K3	platform.kimi.ai